Practice TEST 2

1. What is the difference between a list and a tuple, beyond "one is mutable and one isn't"? Why would you ever choose a tuple?

Answer- A list and a tuple are both used to store multiple values in Python. However, tuples are generally faster and use less memory than lists. Tuples can also be used as dictionary keys, while lists cannot. Lists are suitable when data needs to be changed, whereas tuples are better for fixed data.

2. Why is dictionary lookup O(1) on average? What data structure makes this possible internally?

Answer - Dictionary lookup is O(1) on average because dictionaries are implemented using a hash table. The hash table allows direct access to values through their keys without searching all elements.

3. What does "hashable" mean, and which built-in types are NOT hashable?

Answer- A hashable object is one whose hash value remains constant during its lifetime. Hashable objects can be used as dictionary keys and set elements. Built-in types that are not hashable include list, dictionary, and set because they are mutable.

4. What is the difference between dict.keys() / values() / items() and a list — are they "live" or a snapshot?

Answer- dict.keys(), dict.values(), and dict.items() return live view objects, not lists. They automatically reflect changes made to the dictionary. To get a fixed snapshot, convert them to a list using list().

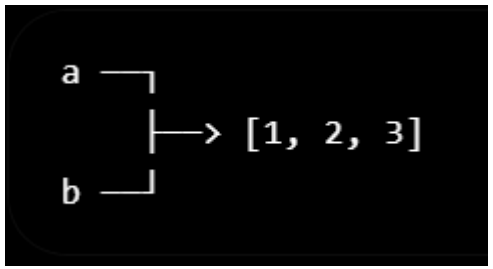
5. Why is set membership testing faster than list membership testing?

Answer- Set membership testing is faster because sets use a hash table, allowing direct access to elements in O(1) average time. Lists require searching elements one by one, which takes O(n) time.

6. What happens in memory when you write b = a where a is a list? Draw or describe it.

Answer- When b = a is executed and a is a list, no new list is created. Both a and b point to the same list object in memory. Therefore, changes made through one variable are reflected in the other.

Memorey draw-



7. What is the difference between a list comprehension and a generator expression — both syntactically and behaviourally?

Answer-A list comprehension uses [ ] and creates all elements immediately in memory. A generator expression uses ( ) and generates elements one at a time when needed. Therefore, generators are more memory-efficient for large amounts of data

Syntax Difference

List Comprehension:

squares = [x*x for x in range(5)]

Generator Expression:

squares = (x*x for x in range(5))

8. Name three things collections.Counter can do that you would otherwise write a manual loop for

Answer- Three tasks that collections.Counter can do are:

1.Count the frequency of elements. 2.ind the most common elements. 3.Count occurrences of words or characters in a string.

These tasks would otherwise require writing manual loops and maintaining counts in a dictionary.

1. Which data structure would you use to remove duplicates from a list while preserving order? _____

Answer- Use a dictionary (or dict.fromkeys()) to remove duplicates while preserving orde

2. Which data structure has guaranteed O(1) average lookup by key? _____

Answer- Dictionary (dict)

3. What error do you get if you try to use a list as a dict key? _____

Answer- TypeError: unhashable type: 'list'

4. What is the output of len({1, 1, 2, 2, 3}) ? _____

Answer- 3

5. What method safely retrieves a dict value without raising KeyError ? _____

Answer- get()

6. What is the time complexity of x in my_set for a set of size n? _____

Answer-O(1) on average

7. Which comprehension type uses curly braces AND a colon? _____

Answer: Dictionary Comprehension

8. What is the namedtuple field access syntax — .field or ["field"] ? _____

Answer- .field

1. Complete to safely get a value with a default of 0

count = my_dict._____("apple", 0)

Answer- count = my_dict.get("apple", 0)

2. Complete to merge two dicts (Python 3.9+ syntax)

merged = dict_a _____ dict_b

Answer- merged = dict_a | dict_b

3. Complete to create an independent copy of a list of lists

import copy deep_copy = copy._____(nested_list)

Answer-

```
import copy
deep_copy = copy.deepcopy(nested_list)
```

4. Complete to count word frequencies in one line

from collections import _____ word_counts = _____(words)

Answer-

```
from collections import Counter
word_counts = Counter(words)
```

5. Complete to create a dict where missing keys auto-initialize to an empty list

from collections import _____ groups = _____(list)

Answer-

```
from collections import defaultdict
groups = defaultdict(list)
```

In [ ]:
```
#Predict the output:
t = (1, 2, 3)
print(t * 2)
print(t + (4,)
```

Answer- (1, 2, 3, 1, 2, 3)

(1, 2, 3, 4)

In [ ]:
```
#Predict the output:
d = {"a": 1, "b": 2}
print(list(d.keys()))
d["c"] = 3
print(d)
```

Answer- ['a', 'b'] {'a': 1, 'b': 2, 'c': 3}

In [ ]:
```
# Predict the output:
s = {1, 2, 3, 2, 1}
print(len(s))
print(s)
```

Answer-

3

{1, 2, 3}

In [ ]:
```
# Predict the output:
lst = [1, 2, 3]
lst2 = lst
lst2.append(4)
print(lst)
print(lst is lst2)
```

Answer- [1, 2, 3, 4] True

In [ ]:
```
# Predict the output:
d1 = {"x": 1}
d2 = {"x": 2, "y": 3}
merged = {**d1, **d2}
print(merged)
```

Answer- {'x': 2, 'y': 3}

In [ ]:
```
#Predict the output:
from collections import Counter
words = ["apple", "banana", "apple", "cherry", "banana", "apple"]
print(Counter(words).most_common(2))
```

Answer- [('apple', 3), ('banana', 2)]

In [ ]:
```
# Predict the output:
matrix = [[0] * 3] * 3
```

```
matrix[0][0] = 1
print(matrix)
```

Answer- [[1, 0, 0], [1, 0, 0], [1, 0, 0]]

In [ ]:
```
 #Predict the output:
def add_item(item, lst=[]):
lst.append(item)
return lst
print(add_item(1))
print(add_item(2))
print(add_item(3))
```

Answer- [1] [1, 2] [1, 2, 3]

In [ ]:
```
# Predict the output:
d = {}
for i in range(3):
d[i] = d.get(i - 1, 0) + 1
print(d)
```

Answer- {0: 1, 1: 2, 2: 3}

Exxplanation- i = 0

d[0] = d.get(-1, 0) + 1 = 0 + 1 = 1

Dictionary:

{0: 1}

i = 1

d[1] = d.get(0, 0) + 1

= 1 + 1

= 2

Dictionary:

{0: 1, 1: 2}

i = 2

d[2] = d.get(1, 0) + 1

= 2 + 1

= 3

Dictionary:

{0: 1, 1: 2, 2: 3}

In [ ]:
```
# Predict the output — and explain WHY:
my_list = [1, 2, 3, 4, 5]
for item in my_list:
if item % 2 == 0:
my_list.remove(item)
print(my_list)
```

Answer- [1, 3, 5]

Reason- The loop iterates over the list while simultaneously modifying it.

In [ ]:
```
# Predict the output — and explain WHY:
a = (1, 2, [3, 4])
a[2].append(5)
print(a)
# Is 'a' still a tuple? Did it just get mutated?
```

Answer- (1, 2, [3, 4, 5])

Reason- a is a tuple, and tuples are immutable, which means you cannot change which objects the tuple contains.

However, the third element of the tuple is a list, and lists are mutable.

In [ ]:
```python
# Predict the output:
keys = ["a", "b", "a", "c"]
values = [1, 2, 3, 4]
result = dict(zip(keys, values))
print(result)
# What happens to duplicate keys?
```

Answer- {'a': 3, 'b': 2, 'c': 4}

Reason- zip(keys, values) creates pairs:

('a', 1) ('b', 2) ('a', 3) ('c', 4)

When dict() creates the dictionary:

'a': 1 is added 'b': 2 is added 'a': 3 overwrites 'a': 1 'c': 4 is added

Since dictionary keys must be unique, a duplicate key replaces the previous value.

The last value wins.

{'a': 3, 'b': 2, 'c': 4}

The first mapping 'a': 1 is overwritten by 'a': 3.

{'a': 3, 'b': 2, 'c': 4}

Reason: Dictionaries do not allow duplicate keys. If the same key appears multiple times, the value from the last occurrence replaces the earlier value

In [ ]:
```python
# This function is supposed to add a discount tag to a list of orders
# WITHOUT modifying the caller's original list. It has a bug — find it.
def apply_discount_tag(orders):
tagged = orders
tagged.append("DISCOUNT_APPLIED")
return tagged
original_orders = ["order_1", "order_2"]
new_orders = apply_discount_tag(original_orders)
print("Original:", original_orders) # Did this change unexpectedly?
```

```
print("New:", new_orders)
# Fix it below:
```

Answer- bug - tagged = orders

This does not create a copy of the list. Both tagged and orders refer to the same list object.

Reason- When the statement

tagged.append("DISCOUNT_APPLIED")

is executed, the original list is also modified because both variables point to the same list

Fix-

```
def apply_discount_tag(orders):
  tagged = orders.copy()      # or orders[:]
 tagged.append("DISCOUNT_APPLIED")
  return tagged


original_orders = ["order_1", "order_2"]
new_orders = apply_discount_tag(original_orders)

print("Original:", original_orders)
print("New:", new_orders)
```

In [ ]:
```
# This is meant to remove all 'inactive' users from a list. It silently
# skips some entries. Find out why and fix it.
users = [
{"name": "Asha", "status": "active"},
{"name": "Ravi", "status": "inactive"},
{"name": "Meena", "status": "inactive"},
{"name": "Karan", "status": "active"},
]
for user in users:
if user["status"] == "inactive":
users.remove(user)
print(users) # Expect 2 active users — count what you actually get
```

```
# Fix it below:
```

Answer- The code removes elements from the list while iterating over the same list.

```
for user in users:
  if user["status"] == "inactive":
    users.remove(user)
```

Reason- Initial list:

```
[Asha(active), Ravi(inactive), Meena(inactive), Karan(active)]
Asha → active → keep
Ravi → inactive → remove
```

List becomes:

```
[Asha(active), Meena(inactive), Karan(active)]
```

The loop now moves to the next position, which contains Karan. Meena is skipped and never checked.

Fix-

```
users = [
{"name": "Asha", "status": "active"},
{"name": "Ravi", "status": "inactive"},
{"name": "Meena", "status": "inactive"},
{"name": "Karan", "status": "active"},
]

users = [user for user in users if user["status"] == "active"]

print(users)
```

In [ ]:
```
# A function is supposed to create independent backups of student
# records so the original roster is untouched. It doesn't work as expected.
roster = [["Asha", [90, 85]], ["Ravi", [70, 75]]]
backup = roster.copy()
```

```
backup[0][1].append(100) # adding a bonus mark to the backup only
print("Roster:", roster) # Did the original get affected too?
print("Backup:", backup)
# Fix it below (hint: which copy function copies ALL levels?):
```

Answer- The bug is:

backup = roster.copy()

copy() creates a shallow copy. It copies only the outer list, but the inner lists are still shared between roster and backup.

Reason- The inner marks lists ([90, 85] and [70, 75]) are referenced by both roster and backup.

So when:

backup[0][1].append(100) the shared inner list changes, affecting both

Fix- import copy

```
roster = [["Asha", [90, 85]], ["Ravi", [70, 75]]]

backup = copy.deepcopy(roster)

backup[0][1].append(100)

print("Roster:", roster)
print("Backup:", backup)
```

In [ ]:
```
# This inventory lookup crashes whenever a product ID isn't found.
# Make it crash-proof without changing the overall logic.
inventory = {"SKU101": 50, "SKU102": 12, "SKU103": 0}
def get_stock(sku):
return inventory[sku]
print(get_stock("SKU101"))
print(get_stock("SKU999")) # <-- crashes here
# Fix it below:
```

Answer- The function uses direct dictionary access:

return inventory[sku]

If the key does not exist, Python raises a KeyError.

Reason- "SKU999" is not present in the dictionary:

inventory = { "SKU101": 50, "SKU102": 12, "SKU103": 0 }

So inventory["SKU999"] crashes.

Fix- inventory = {"SKU101": 50, "SKU102": 12, "SKU103": 0}

```
def get_stock(sku):
  return inventory.get(sku, 0)

print(get_stock("SKU101"))
print(get_stock("SKU999"))
```

In [ ]:
```
# This function matches orders to customer names. It works, but for
# 10,000 orders against 10,000 customers it becomes painfully slow.
# Identify WHY, and rewrite it to be fast at any scale.
orders = [{"order_id": 1, "customer_id": 101}, {"order_id": 2, "customer_id": 102}]
customers = [{"id": 101, "name": "Asha"}, {"id": 102, "name": "Ravi"}]
def attach_customer_name(orders, customers):
for order in orders:
for customer in customers: # <-- the problem is here
if customer["id"] == order["customer_id"]:
order["customer_name"] = customer["name"]
return orders
print(attach_customer_name(orders, customers))
# Rewrite using a dict lookup below (should be O(n + m), not O(n * m)):
```

Answer- The bottleneck is this nested loop:

for order in orders: for customer in customers:

For every order, the code searches through all customers.

If there are:

10,000 orders 10,000 customers

then comparisons ≈

10,000 × 10,000 = 100,000,000

Reason- def attach_customer_name(orders, customers): customer_lookup = {}

```
    for customer in customers:
        customer_lookup[customer["id"]] = customer["name"]

    for order in orders:
        order["customer_name"] = customer_lookup.get(
            order["customer_id"], "Unknown"
        )

    return orders

    print(attach_customer_name(orders, customers))
```

In [ ]:
```
# This is meant to group transactions by (date, category) and accumulate
# totals into a dict keyed by that pair. It throws a TypeError. Why, and fix it?
transactions = [
{"date": "2026-01-01", "category": ["food"], "amount": 200},
{"date": "2026-01-01", "category": ["food"], "amount": 150},
]
totals = {}
for t in transactions:
key = (t["date"], t["category"]) # <-- problem
totals[key] = totals.get(key, 0) + t["amount"]
print(totals)
# Fix it below:
```

Answer- The problem is this line:

key = (t["date"], t["category"])

REASON- Dictionary keys must be immutable and hashable.

These are valid keys:

("2026-01-01", "food") (1, 2) "food"

This is invalid:

("2026-01-01", ["food"])

because it contains a list.

Fix-

```
transactions = [
{"date": "2026-01-01", "category": ["food"], "amount": 200},
{"date": "2026-01-01", "category": ["food"], "amount": 150},
]

totals = {}

for t in transactions:
  key = (t["date"], tuple(t["category"]))
totals[key] = totals.get(key, 0) + t["amount"]

print(totals)
```

In [ ]:
```
# What happens with empty containers? Predict, then run.
print(max([], default="no data"))
print(sum([]))
print({}.get("missing", "default"))
print(list({}.items()))
```

Answer- no data 0 default []

Reason- max([], default="no data") → returns "no data" because the list is empty.

sum([]) → returns 0.

{}.get("missing", "default") → returns "default" because the key is not found.

list({}.items()) → returns [] because the dictionary is empty.

In [ ]:
```python
# Dict literal with duplicate keys — what survives?
d = {"a": 1, "b": 2, "a": 3}
print(d)
# Counter with all-identical items
from collections import Counter
print(Counter(["x", "x", "x"])
```

Answer- {'a': 3, 'b': 2} Counter({'x': 3})

Reason- n a dictionary, duplicate keys are overwritten by the last occurrence. Counter counts the number of occurrences of each item, so "x" is counted 3 times.

In [ ]:
```python
 #Accessing a deeply nested key that may not exist at any level
data = {"user": {"profile": {"address": {"city": "Pune"}}}}
# This crashes if ANY level is missing. Write a safe accessor function
# that returns None instead of raising, for any depth of missing key.
def safe_get(d, *keys):
pass # implement this
print(safe_get(data, "user", "profile", "address", "city"))
print(safe_get(data, "user", "profile", "phone")) # should return None, not crash
```

Answer- def safe_get(d, *keys): current = d

```
    for key in keys:
        if isinstance(current, dict) and key in current:
            current = current[key]
        else:
            return None

    return current
```

In [ ]:
```python
import time
# Compare: membership testing in a list of 200,000 items vs a set
big_list = list(range(200_000))
big_set = set(big_list)
start = time.time()
print(199_999 in big_list)
```

```
print("list lookup:", time.time() - start, "seconds")
start = time.time()
print(199_999 in big_set)
print("set lookup:", time.time() - start, "seconds")
# Run this and observe the gap — this IS the reason sets exist
```

```
True
list lookup: 0.0026237964630126953 seconds
True
set lookup: 8.368492126464844e-05 seconds
```

Answer- True list lookup: (larger time) True set lookup: (much smaller time)

Reason- List Membership (in) 199_999 in big_list

A list searches one element at a time until it finds the target.

Worst case:

0 → 1 → 2 → ... → 199999

In [ ]:
```
# Predict what happens, then run each line one at a time:
try:
s = {1, 2, [3, 4]}
except TypeError as e:
print("Error:", e)
try:
d = {[1, 2]: "value"}
except TypeError as e:
print("Error:", e)
# But a tuple of hashable items works fine:
s2 = {1, 2, (3, 4)}
print(s2)
```

Answer- Error: unhashable type: 'list' Error: unhashable type: 'list' {1, 2, (3, 4)}

Reason- Lists are mutable and therefore not hashable, so they cannot be used as set elements or dictionary keys. Tuples are immutable and hashable (when their contents are hashable), so they can be used safely

In [ ]:
```
# Generators can only be consumed ONCE. Predict the second print:
```

```
gen = (x * x for x in range(5))
print(list(gen))
print(list(gen)) # what do you get the second time? why?
```

[0, 1, 4, 9, 16] []

Reason: Generators are single-use iterators. After the first full iteration, they become exhausted, so the second list(gen) returns an empty list

In [ ]:
```
# A bank needs to flag accounts with more than 3 transactions over ₹50,000
# in a single day, for fraud review. Use a dict to group by account_id.
transactions = [
{"account_id": "ACC1", "amount": 60000},
{"account_id": "ACC1", "amount": 70000},
{"account_id": "ACC2", "amount": 20000},
{"account_id": "ACC1", "amount": 55000},
{"account_id": "ACC1", "amount": 51000},
]
# Your code: build a dict of account_id -> count of transactions > 50000,
# then list accounts where that count > 3
```

Answer-

```
transactions = [
{"account_id": "ACC1", "amount": 60000},
{"account_id": "ACC1", "amount": 70000},
{"account_id": "ACC2", "amount": 20000},
{"account_id": "ACC1", "amount": 55000},
{"account_id": "ACC1", "amount": 51000},
]

fraud_count = {}

for t in transactions:
  if t["amount"] > 50000:
    acc = t["account_id"]
    fraud_count[acc] = fraud_count.get(acc, 0) + 1

   flagged = [acc for acc, cnt in fraud_count.items() if cnt > 3]
```

```
print(fraud_count)
print(flagged)
```

Reason- We use a dictionary to count transactions > 50,000 for each account, then filter accounts where count > 3

In [ ]:
```
# A clinic system stores patient allergies as sets so duplicate entries
# never occur. Two records for the same patient need to be merged safely.
record_a = {"patient": "P1001", "allergies": {"penicillin", "peanuts"}}
record_b = {"patient": "P1001", "allergies": {"peanuts", "latex"}}
# Your code: merge the allergy sets into one combined, de-duplicated set
```

Answer- record_a = {"patient": "P1001", "allergies": {"penicillin", "peanuts"}} record_b = {"patient": "P1001", "allergies": {"peanuts", "latex"}}

```
combined_allergies = record_a["allergies"] | record_b["allergies"]

print(combined_allergies)
```

Reason- Sets automatically remove duplicates.

"|" operator performs set union, which combines both sets into one without repeating elements

In [ ]:
```
# A product catalog has duplicate SKUs from a buggy importer. You need
# to deduplicate by SKU, keeping the FIRST occurrence and preserving order.
products = [
{"sku": "P1", "name": "Mouse"},
{"sku": "P2", "name": "Keyboard"},
{"sku": "P1", "name": "Mouse (duplicate import)"},
{"sku": "P3", "name": "Monitor"},
]
# Your code: deduplicate by 'sku', preserving the first occurrence and order
```

Answer-

```
products = [
{"sku": "P1", "name": "Mouse"},
{"sku": "P2", "name": "Keyboard"},
```

```
{"sku": "P1", "name": "Mouse (duplicate import)"},
{"sku": "P3", "name": "Monitor"},
]

seen = set()
result = []

for item in products:
  if item["sku"] not in seen:
    result.append(item)
    seen.add(item["sku"])

print(result)
```

Reason- We use a set to track seen SKUs -If SKU already exists → skip it

- If not → add to result list
- This preserves first occurrence + order

In [ ]:
```
# HR needs a headcount report: number of employees per department,
# using collections.Counter in a single line.
employees = [
{"name": "Asha", "dept": "Engineering"},
{"name": "Ravi", "dept": "Sales"},
{"name": "Meena", "dept": "Engineering"},
{"name": "Karan", "dept": "HR"},
{"name": "Divya", "dept": "Engineering"},
]
# Your code: build a Counter of department -> headcount
```

Answer-

```
from collections import Counter

employees = [
{"name": "Asha", "dept": "Engineering"},
{"name": "Ravi", "dept": "Sales"},
{"name": "Meena", "dept": "Engineering"},
```

```
{"name": "Karan", "dept": "HR"},
{"name": "Divya", "dept": "Engineering"},
]

dept_count = Counter(emp["dept"] for emp in employees)

print(dept_count)
```

Reason- Counter counts occurrences of each department extracted from the employee list using a generator expression.

In [ ]:
```
# A delivery hub needs to track packages in a FIFO + LIFO hybrid: new
# urgent packages go to the front, regular packages go to the back, and
# a max of 5 packages can be held (oldest gets dropped beyond that).
# Use collections.deque with maxlen.
from collections import deque
dock = deque(maxlen=5)
# Your code: implement add_regular(package) -> append right
# add_urgent(package) -> append left
# then simulate adding 7 packages (mix of regular/urgent) and print final dock
```

Answer-

```
from collections import deque

dock = deque(maxlen=5)

def add_regular(package):
  dock.append(package)

def add_urgent(package):
  dock.appendleft(package)

# Simulating 7 packages (mix of urgent + regular)
 add_regular("P1")
 add_regular("P2")
 add_urgent("P3")
 add_regular("P4")
 add_urgent("P5")
 add_regular("P6")
```

```
add_regular("P7")

print(dock)
```

Reason- Use deque(maxlen=5) with append for regular and appendleft for urgent packages; oldest items are automatically removed when capacity exceeds

"Failure Analysis"

"For each situation, explain what will go wrong and why — before fixing anything"

1. A teammate writes cache = {} as a default function argument: def fetch(key, cache={}): . Six months later, production has a memory leak. Why?

Answer- The dictionary cache={} is created only once when the function is defined, not every time the function is called. Every call to fetch() without providing a cache argument uses the same dictionary object.

Reason- Mutable default arguments are evaluated once at function definition time and then shared across all future calls.

The bug occurs because the programmer expects a new empty dictionary for each call, but Python reuses the same dictionary object every time

Fix-

```
def fetch(key, cache=None):
  if cache is None:
    cache = {}

cache[key] = "value"
return cache
```

2. A reporting script does for k in my_dict: del my_dict[k] to clear a dict during iteration. What happens?

Answer- Python does not allow the size of a dictionary to change during iteration. As soon as an item is deleted, the dictionary's structure changes, making the iterator invalid. A RuntimeError is raised

Reason- When the loop starts, Python creates an iterator over the dictionary's keys. Deleting keys changes the dictionary's size, so the iterator can no longer safely continue.

Fix-

```
for k in list(my_dict.keys()):
  del my_dict[k]
```

3. A junior dev uses a list to track "seen" user IDs across 5 million log lines, checking if uid in seen_list . The job that used to take 2 minutes now takes 6 hours. Diagnose it.

Answer- The problem is that uid in seen_list requires Python to search through the list one element at a time until it finds the ID or reaches the end

Reason- List membership is O(n) (linear time).

With 5 million log lines, the program performs millions of increasingly expensive searches.

Approximate work:

1 + 2 + 3 + ... + 5,000,000 ≈ 12.5 trillion comparisons

This turns what should be a quick job into one that can take hours.

Fix-

```
seen = set()

for uid in log_lines:
  if uid in seen:
    continue
seen.add(uid)
```

4. Someone passes a dict as a default argument to a function that's supposed to return a fresh config each time it's called. What's the risk?

Answer- Suppose a developer writes:

```
def get_config(config={}):
return config
```

They expect each call to return a new empty configuration dictionary.

However, Python creates the default dictionary only once when the function is defined. Every call without an argument gets the same dictionary object

Reason-Mutable default arguments (dict, list, set, etc.) are evaluated once and reused for every call.

The function appears to create a fresh config each time, but actually returns the same shared dictionary repeatedly

Fix-

```
def get_config(config=None):
  if config is None:
    config = {}
return config
```

Hidden Assumptions Each statement below contains a flawed assumption. Identify it.

1. "I'll just use a list — order doesn't matter, and lists are simple.

Answer- Choose the data structure based on the operations you need, not just because it is familiar or simple.

Need ordering/indexing → list Need fast membership/uniqueness → set Need key-value mapping → dict

Flawed assumption: "Simple" automatically means "appropriate." The right data structure depends on the problem's requirements and performance needs.

2. "Dicts in Python don't preserve order, so I always re-sort before printing."

Answer-Use a dict when you want key-value storage and insertion order preservation. Only sort when you specifically need alphabetical or custom ordering.

Flawed Assumption-

"Dicts are unordered." This was true in older Python versions, but in modern Python (3.7+), dictionaries preserve insertion order by default.

3. "Sets and lists support the same operations, just sets are faster.

Answer- Choose based on requirements:

List → ordered collection, duplicates allowed, indexing needed.

Set → unique elements, fast membership tests, order not important.

Flawed Assumption-

"Sets are just faster lists."

They are different data structures with different guarantees, capabilities, and use cases. Speed is only one of the differences.

4. "Deep copy is always safer, so I should always use copy.deepcopy() instead of .copy() just to be safe."

Answer- Use the simplest copy that meets your requirements:

.copy() → when only the top-level container needs to be separate.

copy.deepcopy() → when nested objects must also be independent.

Flawed Assumption

"Safer" does not mean "better in every situation."

deepcopy() is a powerful tool, but it should be used when you specifically need independent copies of nested objects, not as a default habit.

1. You have a function that checks if item in my_list inside a loop that runs 50,000 times, against a list of 10,000 items. Roughly

how many total comparisons is that, and what's the fix?

Answer- A list membership check (item in my_list) is O(n).

In the worst case, Python may need to examine all 10,000 elements for each lookup

Total comparisons:

50,000 × 10,000 = 500,000,000

That's roughly 500 million comparisons.

FIX-

```
# Original list
my_list = [1, 2, 3, 4, 5]

# Convert list to set once
my_set = set(my_list)

# Loop runs 50,000 times
for item in items_to_check:
  if item in my_set:   # Fast O(1) lookup
    print(f"{item} found")
```

2. A defaultdict(list) is being used to group 10 million log lines by user_id . Is this the right structure? What could still go wrong at this scale (think: memory)?

Answer- defaultdict(list) is appropriate for grouping records by user_id. However, storing all 10 million log lines in memory can consume huge amounts of RAM, causing slowdowns or MemoryError. At this scale, data may need to be processed in chunks or written to disk/database instead of keeping everything in memory

3. You're asked to make a "Top 10 most frequent error codes" report from 1 billion log lines that don't fit in memory. Would Counter alone solve this? What's missing from a pure in-memory approach?

Answer- No. Counter only counts frequencies and stores them in memory. Since 1 billion log lines do not fit in memory, the data

must be processed as a stream (line-by-line or in chunks). A pure in-memory approach fails due to memory limits; large numbers of unique error codes can also make the Counter too large

1. Write a function that takes a list of numbers and returns a NEW list

containing only the even numbers, using a list comprehension.

```
def get_evens(numbers):In [ ]:
pass
print(get_evens([1, 2, 3, 4, 5, 6]))
```

In [ ]:
```
Answer-
def get_evens(numbers):
    return [num for num in numbers if num % 2 == 0]

print(get_evens([1, 2, 3, 4, 5, 6]))
```

2. Write a function that takes a list of words and returns a dict

mapping each word to its length.

```
def word_lengths(words):
pass
print(word_lengths(["python", "ai", "ml"]))
```

In [ ]:
```
Answer-
def word_lengths(words):
    return {word: len(word) for word in words}

print(word_lengths(["python", "ai", "ml"]))
```

3. Write a function that returns True if a tuple contains any duplicate values.

```
def has_duplicates(t):
  pass
  print(has_duplicates((1, 2, 3))) # False
  print(has_duplicates((1, 2, 2))) # True
```

In [ ]:
```
Answer-
def has_duplicates(t):
    return len(t) != len(set(t))

print(has_duplicates((1, 2, 3)))
print(has_duplicates((1, 2, 2)))
```

4. Given two lists of equal length representing student names and scores,

build a dict of name -> score, then return the name with the highest score.

```
def top_scorer(names, scores):
pass
print(top_scorer(["Asha", "Ravi", "Meena"], [88, 95, 91]))
```

In [ ]:
```
Answer-
def top_scorer(names, scores):
    student_scores = dict(zip(names, scores))
    return max(student_scores, key=student_scores.get)

print(top_scorer(["Asha", "Ravi", "Meena"], [88, 95, 91]))
```

5. Write a function that flattens a list of lists (one level deep)

using a nested list comprehension.

```
def flatten(list_of_lists):
  passIn [ ]
  print(flatten([[1, 2], [3, 4], [5]])) # [1, 2, 3, 4, 5]
```

In [ ]:
```
Answer-
def flatten(list_of_lists):
    return [item for sublist in list_of_lists for item in sublist]

print(flatten([[1, 2], [3, 4], [5]]))
```

6. Using collections.defaultdict, group a list of words by their first letter.

from collections import defaultdict

```
def group_by_first_letter(words):
  pass
print(group_by_first_letter(["apple", "ant", "banana", "cat", "car"]))
```

{'a': ['apple', 'ant'], 'b': ['banana'], 'c': ['cat', 'car']}

In [ ]:
```
Answer-
from collections import defaultdict

def group_by_first_letter(words):
    groups = defaultdict(list)

    for word in words:
        groups[word[0]].append(word)
```

```
    return dict(groups)

print(group_by_first_letter(["apple", "ant", "banana", "cat", "car"]))
```

7. Write a function that finds all pairs of numbers in a list that sum

to a given target, using a set for O(n) time (not nested loops).

```
    def find_pairs_with_sum(numbers, target):
       pass
       print(find_pairs_with_sum([2, 7, 11, 15, -2, 9], 9)) # should find (2,7) and (11,-2)
```

In [ ]:
```
Answer-
def find_pairs_with_sum(numbers, target):
    seen = set()
    pairs = []

    for num in numbers:
        complement = target - num

        if complement in seen:
            pairs.append((complement, num))

        seen.add(num)

    return pairs

print(find_pairs_with_sum([2, 7, 11, 15, -2, 9], 9))
```

8. Write a function that takes a list of (timestamp, event) tuples and

returns a dict mapping each event type to a deque of its timestamps,

keeping only the last 3 occurrences per event (use collections.deque maxlen).

```
from collections import defaultdict, deque
  def recent_events_by_type(events, keep=3):
    pass
    events = [(1, "login"), (2, "click"), (3, "login"), (4, "login"), (5, "click"), (6, "login")]
print(recent_events_by_type(events))
```

In [ ]:

```
Answer-
from collections import defaultdict, deque

def recent_events_by_type(events, keep=3):
    event_dict = defaultdict(lambda: deque(maxlen=keep))

    for timestamp, event in events:
        event_dict[event].append(timestamp)

    return dict(event_dict)

events = [
    (1, "login"),
    (2, "click"),
    (3, "login"),
    (4, "login"),
    (5, "click"),
    (6, "login")
]

print(recent_events_by_type(events))
```

9. Write a generator function (not a list) that lazily yields running

totals of a list of numbers — without ever storing the full result list.

```
def running_totals(numbers):
  pass
print(list(running_totals([1, 2, 3, 4]))) # [1, 3, 6, 10]
```

In [ ]:
```
Answer-
def running_totals(numbers):
    total = 0
    for num in numbers:
        total += num
        yield total

print(list(running_totals([1, 2, 3, 4])))
```

In [ ]:
```
# Two warehouse systems report stock counts for the same SKUs. Find SKUs
# that DISAGREE between the two systems, and by how much.
warehouse_a = {"SKU1": 100, "SKU2": 50, "SKU3": 20}
warehouse_b = {"SKU1": 100, "SKU2": 45, "SKU4": 10}
# Your code:
# 1. Find SKUs present in only one warehouse
# 2. Find SKUs present in both but with different counts, and the difference
```

Answer- warehouse_a = {"SKU1": 100, "SKU2": 50, "SKU3": 20} warehouse_b = {"SKU1": 100, "SKU2": 45, "SKU4": 10}

```
# 1. SKUs present in only one warehouse
only_in_a = set(warehouse_a) - set(warehouse_b)
only_in_b = set(warehouse_b) - set(warehouse_a)

print("Only in Warehouse A:", only_in_a)
print("Only in Warehouse B:", only_in_b)
```

```
    # 2. SKUs present in both but with different counts
    for sku in set(warehouse_a) & set(warehouse_b):
      if warehouse_a[sku] != warehouse_b[sku]:
        diff = warehouse_a[sku] - warehouse_b[sku]
        print(f"{sku}: A={warehouse_a[sku]}, B={warehouse_b[sku]}, Difference={diff}")
```

In [ ]:

```
# Segment customers into 'high_value' (spent > 10000), 'medium_value'
# (1000-10000), and 'low_value' (< 1000), using a single pass and a dict
# of lists.
customers = [
    {"name": "Asha", "total_spent": 15000},
{"name": "Ravi", "total_spent": 500},
{"name": "Meena", "total_spent": 4500},
{"name": "Karan", "total_spent": 12000},
]
# Your code: build {"high_value": [...], "medium_value": [...], "low_value": [...]}
```

Answer-

```
    customers = [
    {"name": "Asha", "total_spent": 15000},
    {"name": "Ravi", "total_spent": 500},
    {"name": "Meena", "total_spent": 4500},
    {"name": "Karan", "total_spent": 12000},
    ]

    segments = {
    "high_value": [],
    "medium_value": [],
    "low_value": []
    }

    for customer in customers:
      spent = customer["total_spent"]

    if spent > 10000:
        segments["high_value"].append(customer["name"])
    elif spent >= 1000:
```

```
        segments["medium_value"].append(customer["name"])
    else:
        segments["low_value"].append(customer["name"])

    print(segments)
```

In [ ]:

```
 #A web analytics pipeline logs duplicate page-view events due to a
# retry bug. Deduplicate using (user_id, page, timestamp) as the uniqueness
# key, preserving original order, and report how many duplicates were removed.
events = [
{"user_id": "U1", "page": "/home", "ts": 100},
{"user_id": "U1", "page": "/home", "ts": 100}, # duplicate
{"user_id": "U2", "page": "/cart", "ts": 101},
{"user_id": "U1", "page": "/home", "ts": 105},
{"user_id": "U2", "page": "/cart", "ts": 101}, # duplicate
]
# Your code:
```

Answer-

```
events = [
{"user_id": "U1", "page": "/home", "ts": 100},
{"user_id": "U1", "page": "/home", "ts": 100},  # duplicate
{"user_id": "U2", "page": "/cart", "ts": 101},
{"user_id": "U1", "page": "/home", "ts": 105},
{"user_id": "U2", "page": "/cart", "ts": 101},  # duplicate
]

seen = set()
unique_events = []
duplicates_removed = 0

for event in events:
key = (event["user_id"], event["page"], event["ts"])

if key not in seen:
    seen.add(key)
    unique_events.append(event)
else:
```

```
        duplicates_removed += 1

    print("Unique Events:")
    for e in unique_events:
      print(e)

    print("Duplicates Removed:", duplicates_removed)
```

In [ ]:
```
Refactor this to use a dict comprehension instead of a loop
result = {}
for word in ["python", "java", "go"]:
result[word] = len(word)
# Your refactor:
```

Answer-

```
    result = {word: len(word) for word in ["python", "java", "go"]}

    print(result)
```

In [ ]:
```
# Refactor this O(n^2) duplicate-finder to run in O(n) using a set
def has_duplicate(nums):
for i in range(len(nums)):
for j in range(i + 1, len(nums)):
if nums[i] == nums[j]:
return True
return False
# Your refactor:
```

Answer=

```
    def has_duplicate(nums):
    seen = set()

    for num in nums:
        if num in seen:
            return True
        seen.add(num)
```

```
    return False

    print(has_duplicate([1, 2, 3, 4]))     # False
    print(has_duplicate([1, 2, 3, 2]))     # True
```

In [ ]:
```
# Refactor this manual counting loop to use collections.Counter
counts = {}
for tag in ["bug", "feature", "bug", "bug", "chore", "feature"]:
if tag in counts:
counts[tag] += 1
else:
counts[tag] = 1
# Your refactor:
```

Answer-

```
    from collections import Counter

    counts = Counter(["bug", "feature", "bug", "bug", "chore", "feature"])

    print(counts)
```

In [ ]:
```
# This works for the happy path but crashes on empty input or missing keys.
# Refactor to handle both gracefully.
def average_score(students):
total = 0
for s in students:
total += s["score"]
return total / len(students)
print(average_score([])) # currently crashes — fix it
print(average_score([{"name": "Asha"}])) # currently crashes — fix it
```

Answer-

```
    def average_score(students):
    total = 0
    count = 0

    for s in students:
```

```
        if "score" in s:
            total += s["score"]
            count += 1

    if count == 0:
        return 0

    return total / count

    print(average_score([]))
    print(average_score([{"name": "Asha"}]))
    print(average_score([{"name": "Asha", "score": 80},
                     {"name": "Ravi", "score": 90}]))
```

Case: "QuickCart" Flash-Sale Cart Service QuickCart runs a flash sale. Each cart event arrives as: {"cart_id": "C1", "product_id": "P10", "action": "add", "qty": 2} Within minutes, support tickets pour in: customers see duplicate items in their cart, totals are wrong, and the page is slow to load for carts with 50+ items. Analyze — what data structure choices likely caused each symptom? Duplicate items appearing → ? Wrong totals → ? Slow page loads for big carts → ? Design — propose a cart representation (which structure(s) would you use to store cart contents, keyed how?) that prevents all three issues. Implement — write a Cart class/functions using your design: add_item(product_id, qty) , remove_item(product_id) , get_total(price_lookup) .

Your implementation:

class Cart: def **init**(self): pass def add_item(self, product_id, qty): pass def remove_item(self, product_id): pass def get_total(self, price_lookup):

price_lookup: dict of product_id -> price

pass

Answer- Analysis

Duplicate items appearing →

Cart stored as a list of products. Every add event appends a new entry, creating duplicates.

Wrong totals →

Same product appears multiple times with different quantities. Totals are calculated by summing duplicate entries incorrectly.

Slow page loads for big carts →

Using a list requires scanning the entire cart to find/update/remove a product. Operations become O(n), causing delays for large carts. Design

Use a dictionary:

{ product_id: quantity }

Example:

{ "P10": 5, "P20": 2 }

Benefits:

No duplicate product entries.

Quantity updates are straightforward.

Lookup, update, and removal are O(1) average.

code-

```
class Cart:
def __init__(self):
    self.items = {}  # product_id -> quantity

def add_item(self, product_id, qty):
    self.items[product_id] = self.items.get(product_id, 0) + qty

def remove_item(self, product_id):
    self.items.pop(product_id, None)
```

```
    def get_total(self, price_lookup):
        total = 0

        for product_id, qty in self.items.items():
            price = price_lookup.get(product_id, 0)
            total += price * qty

        return total



    cart = Cart()
    cart.add_item("P10", 2)
    cart.add_item("P20", 1)
    cart.add_item("P10", 3)

    prices = {
    "P10": 100,
    "P20": 250
    }

    print(cart.items)
    print("Total =", cart.get_total(prices))

    cart.remove_item("P20")

    print(cart.items)
    print("Total =", cart.get_total(prices))
```

Self-Assessment Checklist

Reflection Section

Reflect in your own words (write directly below each prompt)

1. What pattern(s) did you notice repeating across the Debugging and Edge Case sections today?

Answer- I noticed that many bugs happened because of using the wrong data structure and not considering edge cases. Lists were

often used where sets or dictionaries would be better, causing duplicates and slow performance. I also saw problems with empty inputs, missing keys, and mutable default arguments. The main lesson is to choose the correct data structure and always think about special cases before writing code

2. Which concept took the longest to "click," and what finally made it click?

Answer- The concept that took the longest to click was choosing the right data structure for performance. I initially thought lists could be used for everything, but after solving the scale-thinking and debugging questions, I understood why sets and dictionaries are much faster for lookups and counting. Seeing the difference between $O(n^2)$ and O(n) through examples finally made the concept clear.

3. What's one thing from today you'd explain differently to a friend than how it was explained to you?

ANSWER- I would explain sets and dictionaries using real-life examples instead of time-complexity terms. For example, I would compare a list to searching for a name in a long attendance sheet, while a set or dictionary is like using a roll number index to find information instantly. That makes it easier to understand why the choice of data structure affects performance.

4. What do you want to revise before tomorrow's session?

Answer- Before tomorrow's session, I want to revise Python data structures, especially dictionaries, sets, defaultdict, Counter, and deque. I also want to review time complexity (O(1), O(n), $O(n^2)$) and practice identifying which data structure is best for a given problem. This will help me write more efficient and bug-free code